

# Reliability Maturity Model

The reliability journey is a step-by-step process where each stage builds on the previous one to ensure systems stay available and meet user expectations. This maturity model is intended to help you assess your current state and offer a structured path for improvement.

The foundation begins by bootstrapping basic reliability capabilities offered by Azure by using built-in Azure reliability features like zone redundancy for immediate improvements without extensive optimization overhead.

Counterintuitively, the way to achieve high reliability is to accept failures are inevitable. Rather than trying to prevent every issue, it's more effective to plan how your system will respond when problems occur. Your business requirements help determine which risks are worth addressing proactively. Teams invest in advanced monitoring capabilities with structured observability, extend failure mitigation to include application-level concerns, and begin testing resiliency measures.

Next, teams integrate business insights with technical skills. Teams implement health modeling, conduct failure mode analysis, and prepare comprehensive disaster recovery plans. This stage ensures accountability through measurable objectives and systematic preparation for various failure scenarios.

After the system is live, the emphasis moves to managing the challenges of production environments, including change management and dealing with data growth and operational complexity, and how these affect your system's reliability.

The final level runs indefinitely, and staying resilient is its goal. This level represents the evolution beyond technical controls to architectural adaptability. This level focuses on enabling systems to withstand new and unforeseen risks as workloads evolve and grow.

The model is structured into five distinct maturity levels, each with a primary goal and a set of core strategies. Use the tabbed views below to explore each level. Be sure to also review the highlighted tradeoffs and associated risks as you progress.

## Level 1: Get resilient



**Establish a solid groundwork for resiliency in workload infrastructure and operations, rather than spending time on optimization tasks.**

Level 1 of the maturity model is designed to help workload teams build a strong foundation for system reliability. The focus is on bootstrapping, which is the process of setting up the basics for future reliability decisions. This stage mostly involves functional implementation with minor extensions to current practices.

This stage includes researching, gaining insights, and creating an inventory of your systems. It also uses built-in reliability features on Azure, like enabling zone redundancy for immediate improvements.

By establishing these basics, you can prepare your team to advance through the levels of the reliability maturity model to progressively enhance your system's resilience and performance.

### Key strategies

- ✓ Evaluate opportunities to offload operational responsibility
- ✓ Identify the critical user and system flows
- ✓ Select the right design model, resources, and features
- ✓ Deploy with a basic level of redundancy
- ✓ Enable metrics, logs, and traces to monitor flows
- ✓ Start building a failure mitigation playbook
- ✓ Add mechanisms to recover from transient failures
- ✓ Run basic tests

### ✓ Evaluate opportunities to offload operational responsibility

This strategy is fundamentally a *build* versus a *buy or rely* approach. The decision depends on how much responsibility is manageable at this stage while still supporting future development. You want to use resources that are relevant to the workload, but you should always explore opportunities to offload their maintenance. Here are some classic use cases where you might want to apply this approach.

- **Offload responsibilities to the cloud platform** by choosing platform as a service (PaaS) solutions. They provide ready-made solutions for common resiliency needs like replication, failover, and backup stores. When you take this approach, the cloud provider handles hosting, maintenance, and resilience improvements.

For example, the cloud provider replicates data across multiple compute nodes and distributes the replicas across availability zones. If you build your own solution on

virtual machines (VMs), you need to manage these aspects yourself, which can be time-consuming and complex.

- **Offload responsibilities for operations** that aren't directly tied to the workload's business objectives. Some specialized operations, such as database management and security, can potentially affect the reliability of your workload. Explore the possibility of having experienced teams, technology, or both handle those tasks.

For example, if your team doesn't have database expertise, use managed services to help shift the responsibility to the provider. This approach can be useful when you start out because it allows your team to focus on the functionality of the workload. Many enterprises have shared, centrally managed services. If platform teams are available, use them to handle these operations. However, this approach might add dependencies and organizational complexity.

Alternatively, if your team has the right expertise, you might make an explicit decision to use their skills and select services that don't include management capabilities.

- **Offload responsibilities to non-Microsoft vendors.** Choose off-the-shelf products as the starting point. Build customized solutions only when they contribute to your workload's business value.



**Risk:** If the *buy or rely* option partially fulfills your requirements, you might need to implement custom extensions. This method can result in a "customization lock-in" situation, where updates and modernization become impractical. Regularly review your requirements and compare them with the solution's capabilities. Develop an exit strategy for when there is a significant deviation between the two.

The opposite scenario is also a risk. Although the *buy or rely* option might seem simpler at first, it might require re-evaluation and redesign later if the limitations of the PaaS service, vendor solution, or platform-owned resources don't meet the necessary granularity or level of autonomy needed for the workload.

## ✓ Identify the critical user and system flows

Breaking down the workload into flows is crucial at this stage. Focus on *user* and *system* flows. User flows determine user interactions, and system flows determine communication between workload components that aren't directly associated with user tasks.

For example, in an e-commerce application, customers perform front-end activities like browsing and ordering. Meanwhile, back-end transactions and system-triggered processes fulfill user requests and handle other tasks. Those distinct flows are part of the same system, but they involve different components and serve different purposes.

Start building a catalog of flows at this stage. Observe user interactions and component communication. List and categorize flows, define their start and end points, and note dependencies. Document outcomes and exceptions by using diagrams for clarity. This catalog can serve as an important tool for the initial conversation with business stakeholders to identify the most important aspects from their perspective. This conversation can inform the first level of prioritization.

Classify a flow as critical by evaluating the risk and impact on primary business activities. If you expect an outage, graceful degradation focuses on maintaining these critical flows. In the e-commerce example, critical flows include product searches, adding items to the cart, and checkout because these tasks are essential for business. Other processes, like updating product data and maintaining product images, aren't as critical. Ensure that critical flows remain operational during an outage to prevent revenue loss by allowing users to continue searching for products and adding items to the cart.

#### 📌 Note

A business process can be critical even if it's not time sensitive. Time criticality is a key factor. For example, meeting auditing requirements is a critical process, but you might not need to present data for an audit immediately. The process remains important, but its reliability isn't time critical because recovery within a few hours is acceptable.

For more information, see [Azure Well-Architected Framework: Optimize workload design by using flows](#).

## ✓ Select the right design model, resources, and features

You should apply this strategy at the following levels:

- **Architecture:** The workload design should account for reliability expectations at various infrastructure layers. Your initial decisions might be the choice between containerization or PaaS for hosting the application. Or, you might consider networking setups like hub and spoke or a single virtual network.

You should also set boundaries that create segmentation based on functionality. For example, instead of hosting everything on one VM with a single-zone virtual disk, consider splitting compute and data storage and using dedicated services.

#### ⊗ Caution

In migration scenarios, adopting a lift-and-shift approach without reviewing new opportunities can lead to missed benefits and inefficiencies. It's important to research modernization early to avoid being stuck with setups that are difficult to change and to take advantage of better options and improvements.

- **Azure services:** Use decision trees to help you select the right services for your design. Choose components that meet your current needs, but remain flexible so that you can switch services as your workload evolves and requires more features.
- **SKUs or tiers within Azure services:** Review the features of each SKU and understand the platform's availability guarantees. Evaluate service-level agreements to understand the coverage provided around the published percentile.
- **Features that support reliability:** Choose cloud-native services to enhance availability through simple configurations without changing the code. It's important to understand the options and intentionally select configurations, such as increasing zone redundancy or replicating data to a secondary region.

## ✓ Deploy with a basic level of redundancy

Within each part of your solution, avoid single points of failure, such as single instances. Create multiple instances for redundancy instead. Azure services often handle redundancy for you, especially with PaaS services, which usually include local redundancy by default and options to upgrade. Preferably, use zone redundancy to spread those instances across multiple Azure datacenters. If you don't, at least ensure local redundancy, but this method comes with higher risk. In future levels, you evaluate whether your reliability requirements might be met by extending the solution with geo-redundant components.



**Trade-off:** One significant trade-off is the increased cost of redundancy. Also, cross-zone communication can introduce latency. For legacy applications that require minimal latency, redundancy can degrade performance.



**Risk:** If an application isn't designed for a multiple-instance environment, it might struggle with multiple active instances, which can lead to inconsistent data. Also, if an application is built for an on-premises setup that has low latency, using availability zones might disrupt its performance.

## ✓ Enable metrics, logs, and traces to monitor flows

Choose platform-native tools like Azure Monitor to ensure visibility of metrics, logs, and traces. Use built-in features to set alerts for potential problems. You should have basic alerting in place to send notifications and get alerts. Take advantage of Azure platform capabilities that indicate changes in the health status of services, such as:

- [Azure Service Health](#) to check the platform-level health status.
- [Azure Resource Health](#) to check the resource-level health status.

Set up [Azure Monitor action groups](#) for both the infrastructure and the application.



**Trade-off:** As you collect more logs, you need to manage the increasing volume, which affects the storage-related costs of those logs. Use retention policies to manage the volume. Use Azure Monitor to set a daily cap on a workspace. For more information, see [Configuration recommendations for Reliability](#).

Start building observability at the following layers.

### Infrastructure

Start by enabling diagnostic logs and making sure that you gather native metrics from platform components for analysis. Gather information about resource usage, such as CPU, memory, input/output, and network activity.

### Application

Collect application-level metrics, such as memory consumption or request latency, and log application activities. Do logging operations in a thread or process that's separate from the main application thread. This approach doesn't cause logging to slow down the application's primary tasks.

Also, check the [basic availability tests](#) in Application Insights.

## Data

To monitor databases at a basic level, collect key metrics that the database resources emit. Similar to infrastructure components, track resource usage in the context of data stores, such as networking metrics. Gathering data about how connections are pooled is important for improving efficiency at later stages.

For reliability, it's important to track connection metrics, such as monitoring active and failed connections. For example, in Azure Cosmos DB, a 429 status code is returned when the number of requests exceeds the allocated request units and connections start failing.

### ✓ Start building a failure mitigation playbook

Failures range from intermittent to slightly extended transient failures and catastrophic outages.

In Level 1, focus on platform failures. Even though they're beyond your control, you should still have strategies for handling them. For example, address zone outages by using availability zones. Anticipate transient faults at the platform level and handle them in your workload.

The process of handling these failures varies based on complexity. Start documenting potential platform-level failures, their associated risks, and mitigation strategies. This exercise is primarily theoretical and matures with automation at later levels.

You should document failures, including factors like their likelihood, impact, and mitigation strategies. Use a criticality scale that aligns with your workload's goals. Your scale might include:

- High. A complete system outage that results in significant financial loss and a decline in user trust.
- Medium. A temporary disruption that affects part of the workload and causes user inconvenience.
- Low. A minor software problem that affects a nonessential feature of the application and causes minimal downtime for users.

Here's an example template:

 Expand table

| Problem                                                                   | Risk                                                           | Source                                | Severity | Likelihood                                                     | Mitigation                                                                          |
|---------------------------------------------------------------------------|----------------------------------------------------------------|---------------------------------------|----------|----------------------------------------------------------------|-------------------------------------------------------------------------------------|
| Transient network failure                                                 | The client loses their connection to the application server.   | Azure platform                        | High     | Very likely                                                    | Use design patterns in client-side logic, such as retry logic and circuit breakers. |
| Zone outage                                                               | The user can't reach the application.                          | Azure platform                        | High     | Not likely                                                     | Enable zone resiliency on all components.                                           |
| Transport Layer Security (TLS) certificate expiration                     | The client can't establish a TLS session with the application. | Human error                           | High     | Likely                                                         | Use automated TLS certificate management.                                           |
| CPU or memory usage reaches defined limits and causes the server to fail. | Requests time out.                                             | Application                           | Medium   | Likely                                                         | Implement automatic restarts.                                                       |
| Component is unavailable during an update.                                | The user experiences an unhandled error in the application.    | Deployment or change in configuration | Low      | Highly likely during deployments and not likely at other times | Handle components in client-side logic.                                             |

At Level 1, don't strive for completeness because there are always unforeseen failure cases. If you experience unexpected outages, document the causes and mitigations in the playbook. Treat this asset as a living document that you update over time.

## ✓ Add mechanisms to recover from transient failures

In a cloud environment, transient failures are common. They indicate short-term problems that retries can usually resolve within seconds.

Use built-in SDKs and configurations to handle these faults to keep the system active. Built-in configurations are often the default setting, so you might need to test to validate the implementation. Also, implement patterns that are designed to handle transient failures in your architecture. For more information, see [Architecture design patterns that support reliability](#).



Persistent problems might indicate a failure that isn't transient or the start of an outage. This scenario requires more than just fixing localized problems within the application. It involves examining the critical user and system flows of the system and adding self-preservation techniques and recovery efforts. These methods are mature practices that Level 2 describes.

## ✓ Run basic tests

Integrate basic reliability testing in the early stages of the software development lifecycle. Look for opportunities to do testing, starting with unit tests to validate functionality and configurations.

Also, develop simple test cases for the problems that you identify in the risk mitigation playbook. Focus on higher impact, lower effort mitigations. For example, simulate network outages or intermittent connectivity problems to see how your retry logic resolves the disruptions.



**Risk:** Testing often introduces friction in the development cycle. To mitigate this risk, make reliability testing trackable alongside development tasks.

Feature development is the priority, and testing can introduce friction in the development cycle. It's easier to start testing before feature development is complete. Designing nonfunctional aspects of the application at the beginning allows you to extend them as you add functional capabilities, rather than building up a backlog of problems to address later. Although this approach requires more effort initially, it's manageable and prevents larger problems later.

## Next steps

- Review the [Reliability design review checklist](#) to get details on the recommendations.

Last updated on 07/14/2025